

Aula 11Jogo

Relatório Técnico – Projeto de Redes e Multijogador com Unity

Capa

Nome do Projeto: Mar Made Sushi

Nome do Aluno: Gonçalo Santana de Miranda Pulido Valente

Curso: Desenvolvimento de Jogos Digitais

Data de Entrega:

Numero de Aluno: 40981

Introdução

Breve descrição do objetivo do projeto, tipo de jogo criado e tecnologias envolvidas.

Ideia de jog

Tipo de jogo (FPS, estratégia, por turnos...): Party Game - Estratégia.

Quantos jogadores suporta? 1 a 4 jogadores.

Objetivo principal do jogo: Servir clientes num restaurante para manter uma boa reputação.

Mecânicas principais:

- Cozinhar;
- Servir os clientes;
- Pescar;

Arquitetura de Rede

Sistema utilizado:

☐ Photon | ☐ Mirror | ☒ Netcode | ☐ UTP | ☐ Outro: _____

Tipo de arquitetura:

☒ Peer-to-peer

☐ Host-client

☐ Servidor dedicado

Como os jogadores se ligam entre si?

Inicialmente era com o uso de Relay, em que os jogadores clicavam num unico butao para criar um lobby com um codigo gerado. Os outros jogadores teriam de inserir esse codigo e clicar num botão para se juntarem. Mas infelizmente as instalações onde fiz os testes não permitem esse tipo de ligações, tive de mudar para websockets com conexões por IP e portas. O host cria um lobby num unico clique de um botão e no ecrã aparece um IP. Depois os outros jogadores inserem esse ip no ecrã e pressinam no botão de entrar.

Sincronização

O que está sincronizado? (ex: posição dos jogadores, vida, disparos, objetos)

Posição, animações, objetos, clientes, e cenarios.

Como foi feita a sincronização?

A sincronização foi feita com varias funcionalidades que o Netcode For Gameobject oferece. Como por exemplo, sincronização automatica de posições e animações entre clientes. Mas tambem foram desenvolvidos varios scripts para manter a sincronização entre objectos, clientes, cenarios, etc.

Por RPCs?

Sim, foi usado RPCs para enviar informações de clientes para o servidor, ou até mesmo cliente para servidor e para outros clientes.

Por NetworkVariable?

NetworkVariable foi uma grande ajuda pois facilita muito a sincronização entre clientes sem ter de passar pelo servidor manualmente. Por exemplo, para indicar qual o pedido do cliente que está sentado na mesa, digo atravez de uma networkvariable e por sua vez todos os outros jogados tambem ficam atualizados sobre o pedido.

Com PhotonView?

Nao usei Photon no meu projeto.

Foram usados snapshots, predição, interpolação?

Foi usado interpolação para posições dos jogadores.

Segurança e validação

Que mecanismos foram implementados para evitar erros/trapaças? (ex: verificação de velocidade, limite de dano, controlo no servidor)

Nenhum de momento.

Foi usada encriptação ou filtragem de pacotes?

O unity Netcode For Gameobjects já faz essa encriptação e filtragem por nós desenvolvedores.

Interface e feedback

Que elementos visuais foram usados para mostrar estados? (vida, pontuação, notificações, turnos, etc.)

Pontuação, reações com balões de conversa e vários sprites diferentes. Por exemplo, o tacho de arroz quando não contem nada usa um sprite vazio, quando contem arroz fica a borbulhar e quando cozido fica com arroz a transbordar.

Como é feito o feedback das ações dos jogadores?

Com alteração de sprites ou spawn de objectos.

Testes realizados

Foram feitos testes entre computadores?

Sim, houve varios testes entre computadores na mesma rede e fora.

Quantos jogadores simultâneos foram testados?

Conseguimos juntar 12 jogadores em simultâneo.

Foram simuladas más ligações?

Não.

Algum erro grave foi detetado e corrigido?

Sim, principalmente a sincronização entre objetos foi de certeza um erro grave com muita dificuldade na sua solução.

Desafios enfrentados

Quais foram os principais problemas técnicos encontrados?

Sincronização entre objetos e trabalhar com permissões entre jogadores e o servidor.

Como foram resolvidos?

Para enfrentar estes problemas tive que criar um Network Object Manager que permites os jogadores enviarem mensagens para o servidor a dizer que querem adicionar ou remover um certo objeto. O servidor recebe esse request e cria o novo objecto atribuindo a ownership desse mesmo ao cliente que pediu.

Que partes ficaram por terminar (se houver)?

Fico feliz por dizer que nenhuma parte ficou por terminar, todos os sistemas multiplayer que queria adicionar neste jogo foram concretizados.

Conclusão

O que aprenderam com o projeto?

Apreendi muito sobre como usar o Netcode for Gameobject e permissões entre jogadores e servidor.

O que fariam diferente numa nova versão?

Recriava os meus scripts pois como estava em fase de aprendizagem reparo agora que poderia ter feito de forma diferente e melhor.

Anexos

Printscreen do código mais relevante

```

1  using UnityEngine;
2  using UnityEngine.Networking;
3  using UnityEngine.Events;
4
5  [RequireComponent(typeof(Collider2D))]
6
7  public class MP2D_OnPlayerTrigger : NetworkBehaviour
8  {
9      [SerializeField] private string m_PlayerTag = "Player";
10     [SerializeField] private bool m_ServerOnly = true;
11     [SerializeField] private bool m_LocalPlayerOnly = false;
12     [SerializeField] private KeyCode m_InteractKey = KeyCode.E;
13     [SerializeField] private UnityEvent m_EnterEvents;
14     [SerializeField] private UnityEvent m_ExitEvents;
15     [SerializeField] private UnityEvent m_OnInteractEvents;
16     [SerializeField] private UnityEvent<int> m_EnterEventsWithLocalPlayerId;
17     [SerializeField] private UnityEvent<int> m_ExitEventsWithLocalPlayerId;
18
19     private bool m_IsInsideTrigger = false;
20
21     void OnTriggerEnter2D(Collider2D collision)
22     {
23         HandleTriggerEvent(collision, true);
24     }
25
26     void OnTriggerExit2D(Collider2D collision)
27     {
28         HandleTriggerEvent(collision, false);
29     }
30
31     void Update()
32     {
33         if (!m_LocalPlayerOnly) return;
34
35         if (m_IsInsideTrigger)
36             if (Input.GetKeyUp(m_InteractKey))
37                 m_OnInteractEvents?.Invoke();
38     }
39
40     private void HandleTriggerEvent(Collider2D collision, bool isEnter)
41     {
42         // Check if the colliding object has the correct tag
43         if (!collision.CompareTag(m_PlayerTag)) return;
44
45         // Get the NetworkObject component from the colliding object
46         NetworkObject networkObject = collision.GetComponent<NetworkObject>();
47         if (networkObject == null) return;
48
49         // Handle different multiplayer scenarios
50         if (m_ServerOnly)
51         {
52             // Only execute on the server
53             if (!IsServer) return;
54         }
55         else if (m_LocalPlayerOnly)
56         {
57             // Only execute for the local player
58             if (!networkObject.IsOwner) return;
59             m_IsInsideTrigger = isEnter;
60         }
61         else
62         {
63             // Execute on all clients, but we might want to prevent duplicates
64             // by only executing on the server and then using ClientRpc if needed
65             if (!IsServer) return;
66         }
67
68         // Invoke the appropriate events
69         if (isEnter)
70         {
71             m_EnterEvents?.Invoke();
72             m_EnterEventsWithLocalPlayerId?.Invoke((int)networkObject.OwnerClientId);
73             // Optionally notify all clients about the trigger event
74             if (m_ServerOnly && IsServer)
75             {
76                 OnPlayerTriggerEnterClientRpc(networkObject.OwnerClientId);
77             }
78         }
79         else
80         {
81             m_ExitEvents?.Invoke();
82             m_ExitEventsWithLocalPlayerId?.Invoke((int)networkObject.OwnerClientId);
83             // Optionally notify all clients about the trigger event
84             if (m_ServerOnly && IsServer)
85             {
86                 OnPlayerTriggerExitClientRpc(networkObject.OwnerClientId);
87             }
88         }
89     }
90
91     [ClientRpc]
92     private void OnPlayerTriggerEnterClientRpc(ulong playerId)
93     {
94     }
95
96     [ClientRpc]
97     private void OnPlayerTriggerExitClientRpc(ulong playerId)
98     {
99     }
100 }

```

```

33     // This will be called on all clients when a player enters the trigger
34     // You can add client-specific logic here if needed
35     // For example: UI updates, sound effects, etc.
36 }
37
38 [ClientRpc]
39 1 reference
40 private void OnPlayerTriggerExitClientRpc(ulong playerId)
41 {
42     // This will be called on all clients when a player exits the trigger
43     // You can add client-specific logic here if needed
44 }
45
46 // Alternative method using ServerRpc if you want clients to report trigger events to server
47 [ServerRpc(RequireOwnership = False)]
48 0 references
49 private void ReportTriggerEventServerRpc(bool isEnter, ulong reportingClientId)
50 {
51     // Handle the trigger event on the server
52     if (isEnter)
53     {
54         m_EnterEvents?.Invoke();
55     }
56     else
57     {
58         m_ExitEvents?.Invoke();
59     }
60 }
61
62 }

```

Foi o script mais utilizado no jogo pois permitia cada jogador interagir com qualquer parte do cenário.

Link para repositório (GitHub, Drive...)

<https://github.com/Valente-Coding/ModularMultiplayer>

Vídeo de demonstração (se aplicável)

<https://youtu.be/4pTJCFrdATc>